

Technical Execution Plan

Epic 7084: Azure Entra ID SSO Integration

System	Intervention Management Platform
Target Audience	Technical Management & Engineering Team
Estimated Duration	4 Weeks (2 Sprints)
Resource Allocation	1 Senior Engineer

1. Architectural Design Strategy

Before initiating the technical execution, the following core design principles will be enforced to guarantee an enterprise-grade delivery for the Intervention Management Platform:

- **Zero-Trust Security Baseline (Story 7091):** Hardcoded secrets will be strictly prohibited. Azure Key Vault will be injected into the ASP.NET Core configuration pipeline at startup via Azure Managed Identities.
- **Decoupled Authentication Pipeline (Stories 7089, 7093, 7095):** The UI application will exclusively handle identity acquisition (OIDC). The API will remain completely stateless, validating JWT Bearer tokens. Token transmission will be centralized via a `DelegatingHandler` to maintain Clean Architecture.
- **Non-Destructive Authorization (Story 7092):** The existing database-driven RBAC will not be replaced. Instead, an `IClaimsTransformation` layer will map incoming Entra ID Object IDs to the existing legacy database roles dynamically.
- **Zero-Downtime Rollout (Story 7096):** ASP.NET Core feature flags mapped to Azure App Service Slot settings will enable a "Dark Launch." The team can continue using local logins while SSO is validated in parallel.

2. Implementation Plan & Story Mapping

To minimize critical path blockers, the execution is structured into four sequential phases across a 4-week timeline.

Phase 1: Security Foundation & UI Auth (Week 1)

Objective: Establish the secure perimeter and prove UI login capabilities.

ID	User Story	Technical Execution Strategy
7091	Azure Key Vault Integration	Provision Key Vaults for target environments. Implement <code>Azure.Extensions.AspNetCore.Configuration.Secrets</code> in both UI and API host builders.
7089	Implement UI Entra ID Login	Configure <code>Microsoft.Identity.Web</code> . Implement the dual-authentication feature flag logic inside the MVC <code>AccountController</code> .
7090	Integrate Microsoft Graph	Configure Graph API client to fetch the user profile picture and basic details post-login for population in the UI navbar.

Phase 2: API Protection & Token Transmission (Week 2)

Objective: Lock down the backend and establish secure UI-to-API communication.

ID	User Story	Technical Execution Strategy
7093	Secure API with JWT Validation	Add <code>JwtBearer</code> authentication. Configure dynamic scheme selection to seamlessly support both local JWTs and Entra ID tokens.
7095	UI-to-API Token Transmission	Implement <code>TokenAuthorizationHandler</code> middleware on the UI to securely attach Bearer tokens to all outbound HTTP client requests.
7094	Enable Swagger UI Auth	Configure OAuth2 PKCE flow in SwaggerGen options to allow developers to securely invoke and test protected API endpoints.

Phase 3: Authorization & RBAC Mapping (Week 3)

Objective: Ensure existing business rules and permissions remain perfectly intact.

ID	User Story	Technical Execution Strategy
7092	Integrate SSO with DB RBAC	Implement <code>IClaimsTransformation</code> in the API. Extract Entra oid, query the cached database repository, and append existing roles to the <code>ClaimsPrincipal</code> .

Phase 4: Environment Rollout & Validation (Week 4)

Objective: Pipeline configuration, slot deployments, and final QA sign-off.

ID	User Story	Technical Execution Strategy
7096	Configure SSO Across Envs	Setup App Registrations for Dev, UAT, Staging, PreProd, and Prod. Configure App Service Slot settings to toggle the SSO feature flag dynamically.
7097	Perform End-to-End Testing	Validate dual-login persistence, token expiration/refresh cycles, and guarantee API rejection of invalid audiences or scopes.
7086	Integration Readiness	Final code review, architectural documentation updates, and execution of merge to the main branch.

3. Key Risks & Mitigation Strategies

The following risks have been identified and must be proactively managed to ensure delivery on schedule.

1. Entra ID Admin Consent Bottleneck

Story 7090 (Graph API) requires `User.Read` and `Mail.Send` delegated permissions. In enterprise Azure tenants, granting these often requires global IT Admin consent.

Mitigation: Raise IAM tickets and request these App Registration permissions immediately in Sprint 1 to prevent stalling the execution phase.

2. App Service TLS Termination

Azure App Service load balancers terminate HTTPS traffic. The underlying Kestrel server will receive HTTP requests, causing MSAL/Entra ID to reject the redirect URIs as insecure.

Mitigation: The UI application pipeline must be explicitly configured with `UseForwardedHeaders` middleware to preserve the original protocol scheme.

3. Local Development Disruption

Altering the authentication flow can block other developers actively building features.

Mitigation: The feature flag architecture strictly isolates the SSO code. Local environments will default to `EnableSSO = false`, allowing the team to function normally via legacy database authentication.